Regresión Simbólica para Descubrir Leyes Físicas

Jorge Acebes Hernández, Andrés López Serna y Lorenzo Ji

Grado en Física

Resumen

Este proyecto estudia la capacidad de distintos modelos de aprendizaje automático para descubrir leyes físicas a partir de datos experimentales. Se comparan regresores simbólicos con perceptrones multicapa (MLP) evaluando la bondad de su ajuste, su interpretabilidad y su capacidad de generalización. Los resultados muestran la dominancia de los regresores simbólicos PySR y GPLearn, capaces de recuperar la expresión analítica correctamente para la gran mayoría de casos, independientemente del nivel de ruido. El código del estudio se encuentra en GitHub [1], pudiendo ejecutar los modelos tanto desde el código fuente (`src`) como desde el front-end interactivo (`web`).

1. Introducción

La identificación de leyes matemáticas que rigen las variables físicas constituye el formalismo esencial para la modelización y comprensión del universo. Históricamente, la obtención de estas relaciones ha operado bajo un paradigma de retroalimentación entre la deducción teórica y la validación empírica. No obstante, la actual convergencia entre la disponibilidad masiva de datos y el incremento en la capacidad de cómputo ha habilitado una vía alternativa: la inferencia directa de estructuras analíticas a partir de la observación experimental.

En este contexto, la regresión simbólica se posiciona como una técnica fundamental al trascender el mero ajuste paramétrico; su objetivo es la búsqueda de la topología funcional que gobierna el sistema. A diferencia de otros métodos de aprendizaje automático como las redes neuronales, que operan como aproximadores universales de naturaleza opaca o "caja negra", la regresión simbólica optimiza el compromiso entre precisión predictiva y parsimonia. Esta interpretabilidad estructural es indispensable en la praxis científica, pues permite que el modelo inferido sea validado teóricamente y generalizado a nuevos dominios físicos.

Este trabajo analiza diversas metodologías orientadas al redescubrimiento de leyes físicas a partir de datos, evaluando algoritmos de regresión simbólica frente a perceptrones multicapa (MLP) y regresión polinómica. El estudio compara su capacidad para extraer leyes ocultas y su robustez ante el ruido mediante el uso de datos sintéticos con una distribución gaussiana $\mathcal{N}(0, \sigma^2)$. Bajo este marco experimental, se analiza la capacidad de cada método para satisfacer el principio de parsimonia, transformando la mera precisión predictiva en una herramienta de inferencia estructural. En última instancia, esta comparativa permite explorar cómo la sinergia entre el aprendizaje automático y la física computacional puede trascender el ajuste de modelos para actuar como un catalizador en la expansión del conocimiento científico fundamental.

2. Generación de datos

Los conjuntos de datos empleados se generan sintéticamente a partir de las leyes físicas detalladas en la Tabla 1. Cada conjunto consta de 10^4 muestras, cuyas variables independientes se extraen de distribuciones uniformes aleatorias acotadas en rangos físicamente admisibles. Para permitir que el algoritmo se centre exclusivamente en la topología de la ley, las variables se adimensionalizan forzando a que todas las constantes del sistema equivalgan a la unidad. La coherencia en el Sistema Internacional se garantiza a posteriori mediante reescalados de los coeficientes o transformaciones afines, cuya estructura viene estrictamente dictada por el análisis dimensional y las simetrías de la ley en cuestión.

Fenómeno Físico	**Ecuación**
Ley de Coulomb	$F = q_1 q_2 / r^2$
Oscilador armónico	$F = -x$
Tercera ley de Kepler	$T = r^{3/2}$
Ley de los gases ideales	$P = nT/V$
Dilatación temporal	$t' = t/\sqrt{1-v^2}$
Rango proyectil	$R = v_0^2 \sin(2\theta)$
Decaimiento radiactivo	$N = e^{-\lambda t}$
Enfriamiento de Newton	$\Delta T = 1 + e^{-kt}$
Entropía de Boltzmann	$S = \ln \Omega$

Tabla 1: Leyes fundamentales en unidades normalizadas.

Con el objetivo de simular la incertidumbre intrínseca de las medidas experimentales, se introduce ruido blanco gaussiano proporcional a la dispersión de los datos. El proceso de perturbación se define mediante la expresión:

$$y_{\text{noisy}} = y + \varepsilon \cdot \sigma_y \cdot \mathcal{N}(0,1), \tag{1}$$

en la que y representa la variable objetivo, ε el nivel de ruido prefijado y σ_y la desviación estándar de dicha variable calculada sobre la totalidad del conjunto de datos de la correspondiente expresión. El término $\mathcal{N}(0,1)$ denota una variable aleatoria extraída de una distribución normal estándar. Este enfoque garantiza que la perturbación sea adimensionalmente consistente y proporcional a la escala de los datos. Para el análisis de robustez, se evalúan tres regímenes de control: ausencia de ruido ($\varepsilon = 0$), ruido bajo ($\varepsilon = 0.01$) y ruido alto ($\varepsilon = 0.1$).

Por último, para cada ley se genera un *dataset* adicional empleando variables fuera del rango previamente estudiado (*outside of domain*, OOD) y en ausencia de ruido. Esto posibilitará evaluar la capacidad que tienen los distintos modelos para generalizar la física fuera del rango de entrenamiento.

3. Preprocesamiento de datos

Antes de entrenar los modelos, los datos se preprocesan para poder obtener resultados fiables y precisos. En particular, se eliminan valores no válidos (`NaN` e `inf`), garantizando la estabilidad del entrenamiento, y se realiza una partición de cada *set* de muestras de la forma 60/20/20 (60 % entrenamiento, 20 % validación, 20 % test). Tras realizar el *split*, se aplica una transformación tipo z-score para los modelos MLP y de regresión polinómica:

$$x' = \frac{x - \mu}{\sigma}, \tag{2}$$

siendo los parámetros μ (media) y σ (desviación estándar) calculados exclusivamente sobre el conjunto de entrenamiento, previniendo el *data leakage*.

El motivo por el cual se realiza este proceso de estandarización de los datos para los modelos mencionados es el aumento de su estabilidad, así como una mejoría en la competencia de los datos en el caso particular de la regresión polinómica. En el caso de los modelos de regresión simbólica este preprocesado no es tan relevante, no obstante, algunos métodos como QLattice poseen procesos automáticos de codificado de los argumentos de entrada para mejorar su rendimiento [3].

4. Funciones de coste y evaluación

4.1. Funciones de coste

La mayoría de los modelos empleados se optimizan, directa o indirectamente, mediante una pérdida cuadrática. En particular, la métrica dominante es el error cuadrático medio, definido como:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \tag{3}$$

siendo y_i el valor real, $\hat{y}_i$ la predicción del modelo y N el número de muestras. No obstante, algunos modelos implementan otras funciones de coste y regularizaciones adicionales, recogidos en la Tabla 2.

Modelo	**Pérdida principal**	**Criterio adicional**
MLP estándar	MSE	–
MLP sparse	MSE	Penalización L_1 sobre los parámetros
MLP dropout	MSE	*Dropout* como regularización arquitectónica
Regresión polinomial	Mínimos cuadrados	–
PySINDy	Mínimos cuadrados secuenciales	Umbralización de coeficientes
GPLearn	MSE	Coeficiente de parsimonia
PySR	Error cuadrático	Parsimonia y selección por complejidad
QLattice	Error cuadrático	*Bayesian Information Criterion* (BIC)

Tabla 2: Resumen de las funciones de coste y criterios de regularización empleados.

4.2. Función de evaluación

La bondad del ajuste se evaluará mediante el error cuadrático medio, dado por la Expresión 3. En función del valor de MSE obtenido, la regresión se clasifica en uno de estos cuatro grupos: Exacta ($\mathrm{MSE} < 10^{-20}$), Precisa ($10^{-20} \leq \mathrm{MSE} < 10^{-4}$), Aproximada ($10^{-4} \leq \mathrm{MSE} < 10^{-2}$) o Incorrecta ($\mathrm{MSE} \geq 10^{-2}$).

5. Modelos utilizados

5.1. Regresión polinómica

La regresión polinómica es una extensión de la regresión lineal para obtener relaciones no lineales mediante la adición de términos de una base de variables en forma de potencias con exponentes enteros

desde 1 hasta D. De esta forma, la variable objetivo se aproxima por un conjunto de potencias y coeficientes hasta un grado máximo D.

En este proyecto, la expansión polinómica se realiza empleando la función `PolynomialFeatures` de `scikit-learn` [6]. Partiendo de un vector de entrada de n variables diferentes: $\vec{x} = (x_1, x_2, \ldots, x_n)$, se define un nuevo vector:

$$\vec{\phi}(\vec{x}) = \left(\prod_{j=1}^{p} x_j^{k_j} \right), \quad \text{con} \quad \sum_{j=1}^{p} k_j \leq D, \quad k_j \in \mathbb{N}. \tag{4}$$

Así, la variable objetivo se puede ajustar de la siguiente forma:

$$\hat{y} = \hat{\beta}_0 + \hat{w}^T \vec{\phi}(\vec{x}), \tag{5}$$

donde $\hat{w}$ contiene los coeficientes del ajuste.

La regresión polinómica ofrece relaciones analíticamente interpretables, pero presenta dos limitaciones fundamentales. En primer lugar, el número de términos escala de forma combinatoria, generando expresiones demasiado complejas para una interpretación intuitiva en espacios de alta dimensión. En segundo lugar, para modelar relaciones físicas no polinómicas como exponenciales, logaritmos o raíces, el algoritmo recurre a aproximaciones locales, que pierden precisión rápidamente en rangos amplios y se vuelven inmanejables al aumentar el número de variables.

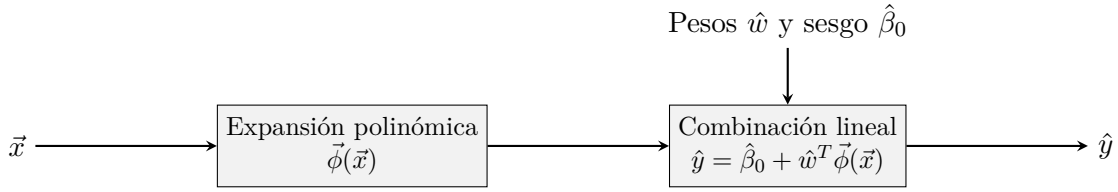

Figura 1: Diagrama de flujo de una regresión polinómica clásica.

5.2. Redes neuronales multicapa (MLP)

Las redes neuronales multicapa o perceptrones multicapa (MLP) ajustan sus parámetros para modelar relaciones altamente no lineales, sacrificando interpretabilidad analítica. Estructuralmente, la red procesa la información a través de capas ocultas intermedias totalmente conectadas (*fully connected*), las cuales proyectan los datos a nuevos espacios de representación mediante transformaciones afines seguidas de no linealidades para extraer características abstractas.

En el marco de este estudio, los distintos MLPs fueron implementados en `PyTorch` y entrenado por lotes (*batches*) mediante descenso de gradiente estocástico (`Adam`).

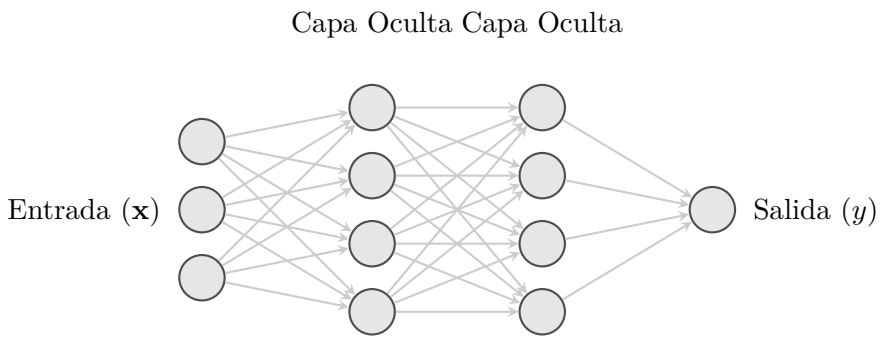

Figura 2: Diagrama general de la arquitectura de un Perceptrón Multicapa (MLP).

5.2.1. MLP estándar

El modelo estándar consta de dos capas ocultas: $d_{\text{in}} \to 64 \to 64 \to 1$ y cuya función de activación es SiLU (*Sigmoid Linear Unit*). Esta función se define como:

$$\text{SiLU}(x) = x \cdot \sigma(x).$$

Se trata de una función continua de clase C^∞, es decir, es infinitamente derivable, lo que evita la aparición de discontinuidades en los gradientes. En la Figura 3 se puede apreciar tanto la arquitectura de la red, como el comportamiento de la función de activación.

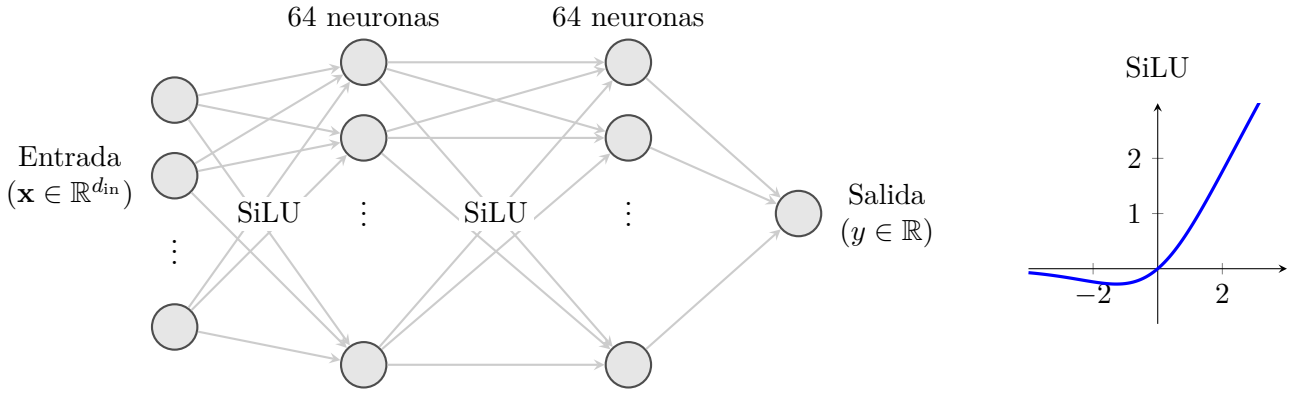

Figura 3: Arquitectura MLP $d_{\text{in}} \to 64 \to 64 \to 1$. Las proyecciones afines hacia las capas ocultas aplican la función de activación continua $\text{SiLU}(x) = x \cdot \sigma(x)$, mostrada en el panel derecho. La proyección hacia la capa de salida es puramente lineal.

5.2.2. MLP con regularización L1 (Disperso)

Con el objetivo de introducir una estructura más simple estructuralmente, se considera una variante de la MLP estándar con un menor número de neuronas por capa. Su arquitectura es: $d_{\text{in}} \to 16 \to 16 \to 1$, y emplea la función de activación $\tanh(x)$.

Adicionalmente, durante el entrenamiento, se añade un término de regularización L_1 sobre todos los parámetros del modelo:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda \sum_i |w_i|. \tag{6}$$

Este término induce *sparsidad*, provocando que aquellos pesos con menor relevancia tomen valores cercanos a 0. De esta manera, se fuerza al modelo a utilizar un menor número de conexiones y, consecuentemente, deriva en un modelo más simple, generalista y resistente al sobreajuste. Pese a la simplificación paramétrica, el modelo no es comparable con los regresores simbólicos en cuanto a la interpretabilidad. Su estructura se encuentra reflejado en la Figura 4.

5.2.3. MLP con Monte Carlo Dropout

Para dotar al modelo de capacidad predictiva probabilística y estimar su incertidumbre epistémica, se implementa *Monte Carlo Dropou*t (MC Dropout). La arquitectura empleada es $d_{\text{in}} \to 32 \to 32 \to 1$, utilizando activaciones ReLU y aplicando *dropout* con probabilidad $p = 0.2$ tras las dos capas ocultas.

A diferencia de su uso tradicional, el *dropout* se mantiene activo durante la fase de inferencia. Para una entrada dada, se realizan M pases estocásticos hacia adelante, evaluando equivalentes a M

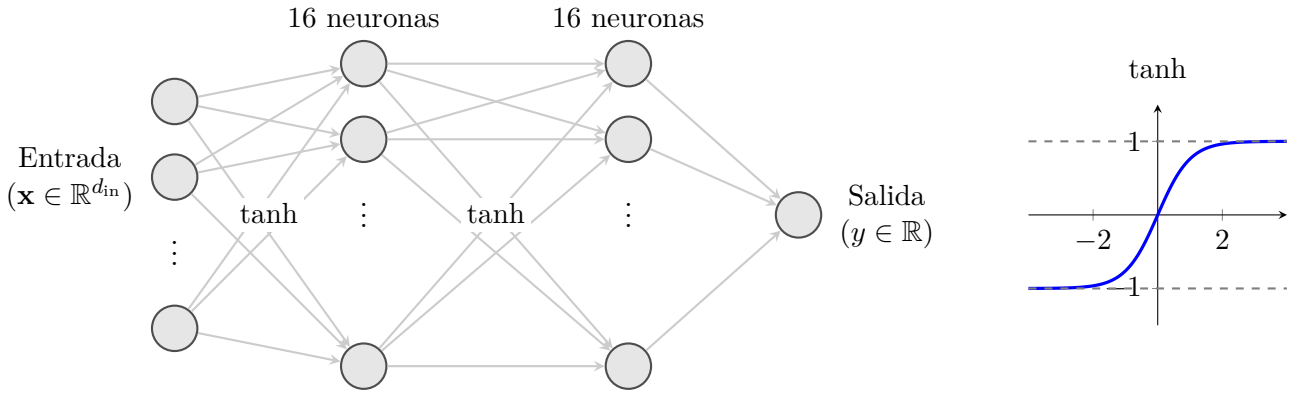

Figura 4: Arquitectura MLP $d_{\text{in}} \to 16 \to 16 \to 1$. Las proyecciones afines hacia las capas ocultas aplican la función de activación continua $\tanh(x)$, mostrada en el panel derecho. El modelo se entrena bajo regularización L_1 para inducir dispersión en los pesos de las conexiones representadas.

subredes distintas. Esto permite aproximar la distribución predictiva marginal, obteniendo la predicción final como la media empírica de las salidas,

$$\mathbb{E}[y] \approx \frac{1}{M} \sum_{i=1}^{M} \hat{y}_i,$$

y cuantificando la incertidumbre epistémica del modelo a través de su varianza muestral,

$$\mathrm{Var}[y] \approx \frac{1}{M} \sum_{i=1}^{M} (\hat{y}_i - \mathbb{E}[y])^2.$$

El esquema correspondiente a la arquitectura del *Monte Carlo Dropout* junto con su función de activación se presenta en la Figura 5.

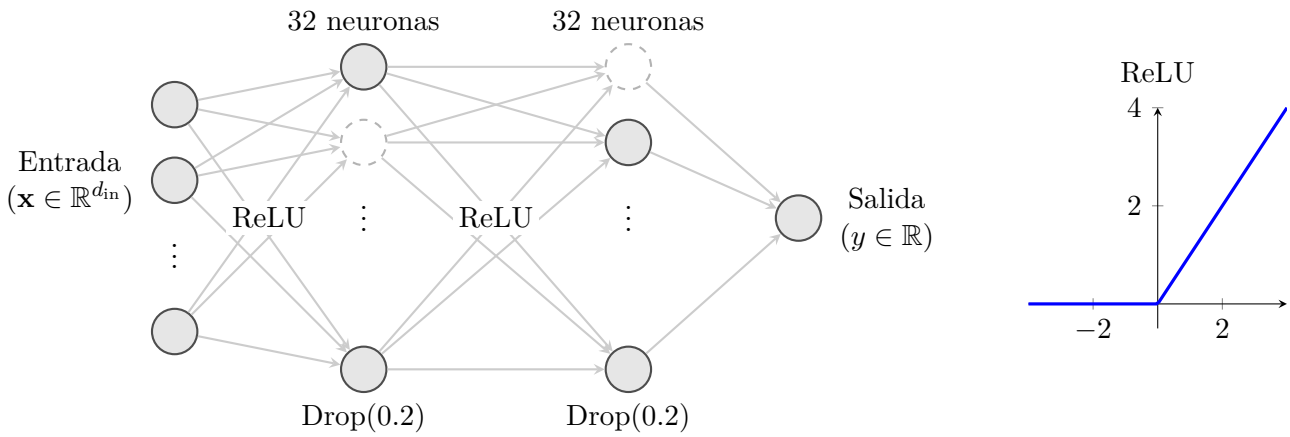

Figura 5: Arquitectura MLP $d_{\text{in}} \to 32 \to 32 \to 1$ con Monte Carlo Dropout. Las proyecciones hacia las capas ocultas aplican la activación $\mathrm{ReLU}(x) = \max(0, x)$. Se incluyen capas de *dropout* ($p = 0.2$) tras cada capa oculta. Los nodos punteados ilustran la desactivación estocástica de neuronas durante una pasada de inferencia, mecanismo que permite aproximar la incertidumbre epistémica del modelo.

5.3. Regresión simbólica

La regresión simbólica explora el espacio de expresiones matemáticas para encontrar la función $f(\mathbf{x})$ que minimice la función de coste respecto a los datos experimentales.

5.3.1. PySINDy

PySINDy (*Sparse Identification of Nonlinear Dynamics*) [5], concebido originalmente para la extracción de ecuaciones diferenciales, se implementa en este contexto como un regresor simbólico fundamentado en la optimización dispersa sobre un diccionario de funciones elementales. El modelo aproxima la variable objetivo mediante una combinación lineal:

$$y \approx \Theta(X)\boldsymbol{\xi},$$

donde $\Theta(X)$ es la matriz de diseño que evalúa la biblioteca de funciones sobre los datos empíricos, y $\boldsymbol{\xi}$ es el vector de coeficientes a determinar.

La matriz $\Theta(X)$ define un espacio funcional que amalgama términos polinómicos y racionales. La base polinómica, truncada a un grado máximo d (hiperparámetro que acota la complejidad topológica), incluye potencias e interacciones cruzadas ($\{x_i, x_i^2, x_i x_j, \dots\}$). Para modelar las dependencias inversas inherentes a múltiples leyes físicas, se adjuntan funciones racionales regularizadas: $\{(x+\epsilon)^{-1}, (x^2+\epsilon)^{-1}\}$, donde $\epsilon = 10^{-8}$ actúa como un factor de estabilidad numérica frente a singularidades en el entorno del cero.

El cálculo del vector $\boldsymbol{\xi}$ se delega al algoritmo STLSQ (*Sequential Thresholded Least Squares*). Este método iterativo resuelve un problema de mínimos cuadrados con penalización L_2 (regularización *Ridge*), aplicando simultáneamente un truncamiento que anula todo coeficiente inferior a un umbral de corte (`threshold`). La convergencia del algoritmo garantiza la retención exclusiva de los términos dinámicos dominantes.

Tras el ajuste, la expresión analítica emerge como la superposición de los términos activos ($\mathcal{S}$):

$$y \approx \sum_{i \in \mathcal{S}} \theta_i(X)\xi_i$$

Si bien la linealidad de esta combinación garantiza una alta interpretabilidad, revela la principal vulnerabilidad del método: la dependencia absoluta del sesgo inductivo del diccionario. Si la fenomenología subyacente requiere operadores excluidos del espacio generador de $\Theta(X)$, el algoritmo sacrifica la dispersión para forzar la minimización de la pérdida, densificando $\boldsymbol{\xi}$ y derivando inevitablemente en el sobreajuste del modelo.

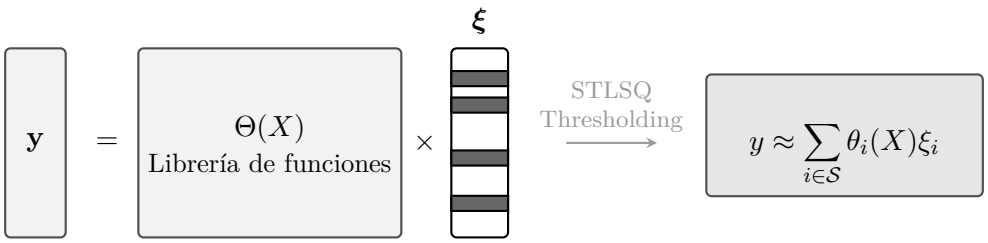

Figura 6: Representación esquemática del método PySINDy. A partir de la matriz de diseño $\Theta(X)$ y el vector de coeficientes disperso $\boldsymbol{\xi}$, el algoritmo STLSQ anula los elementos cercanos a cero y aísla el conjunto de términos activos $\mathcal{S}$.

5.3.2. GPLearn

GPLearn [7] implementa la regresión simbólica mediante programación genética pura. A diferencia de los enfoques fundamentados en diccionarios estáticos, el algoritmo modela el espacio de búsqueda topológicamente mediante árboles de sintaxis abstracta, donde los nodos internos operan como funciones matemáticas y las hojas actúan como variables independientes o constantes.

En este estudio, su espacio funcional viene generado por el conjunto de operadores $\{+, -, \times, \div, \sin x, \cos x, x^{-1}, \sqrt{x}, \ln x\}$. Esta formulación evita la restricción a formas polinómicas y dota al modelo de la flexibilidad necesaria para reconstruir relaciones físicas composicionales y no lineales complejas.

La dinámica evolutiva arranca con una población de expresiones generadas estocásticamente. La supervivencia se rige por un proceso de selección por torneo, donde prevalecen los individuos que minimizan el error predictivo. Antes de transitar a la generación sucesiva, la población se somete a operadores genéticos controlados estocásticamente por hiperparámetros. Estos mecanismos incluyen la recombinación o *crossover* (intercambio de subárboles entre progenitores), mutaciones estructurales y puntuales (alteración total o parcial de nodos), mutaciones de contracción o *hoist* (simplificación topológica eliminando ramas superfluas) y la reproducción directa. La envergadura de la exploración queda así determinada por el tamaño de la población y el número máximo de generaciones.

Para garantizar la interpretabilidad y mitigar el sobreajuste, la optimización no se limita a minimizar el error empírico. La función de coste incorpora un término de regularización denominado coeficiente de parsimonia (*parsimony coefficient*). Esta métrica introduce una penalización matemática directamente proporcional a la complejidad morfológica del árbol (número de nodos y profundidad), forzando el colapso sistemático de la red hacia expresiones compactas.

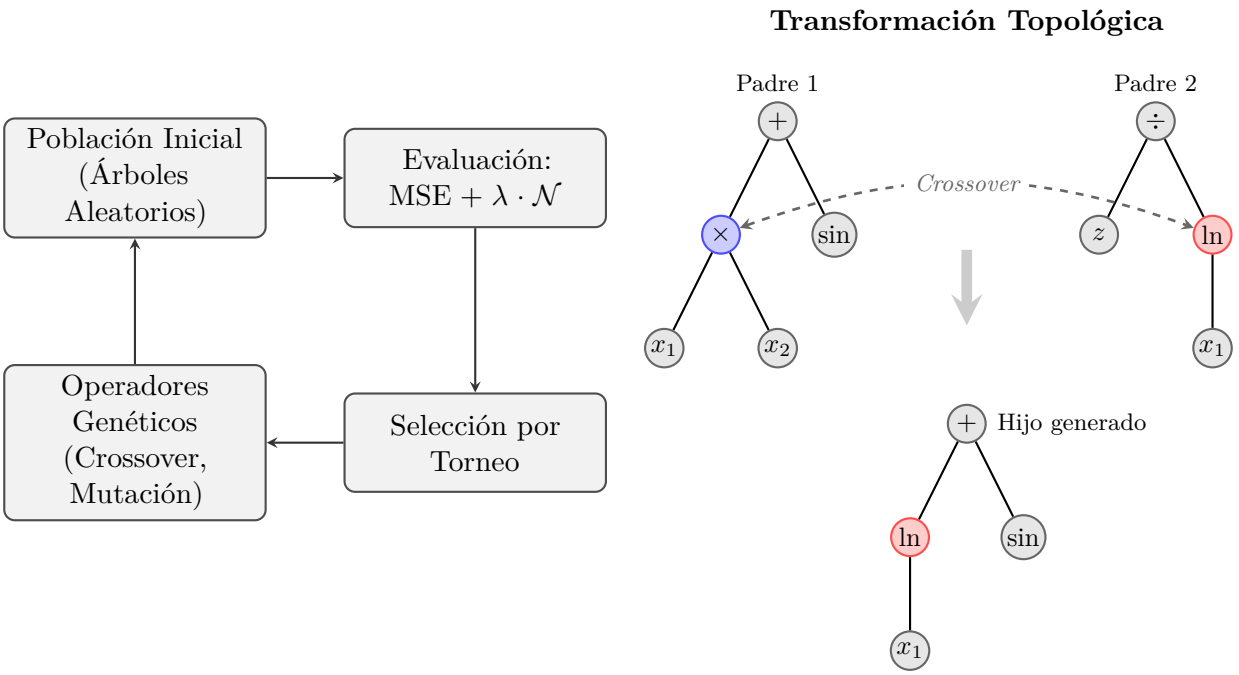

Figura 7: Dinámica de GPLearn. A la izquierda, flujo iterativo de la población bajo presión de parsimonia. A la derecha, detalle del operador de *crossover*: el subárbol destacado del Padre 2 reemplaza genéticamente al subárbol del Padre 1, derivando en una nueva topología funcional.

5.3.3. PySR

PySR (*Symbolic Regression via Evolutionary Algorithms*) [4] es una herramienta de regresión simbólica optimizada para el descubrimiento de leyes físicas a partir de datos empíricos. Su arquitectura maximiza el rendimiento computacional delegando la exploración del espacio de expresiones matemáticas a un *backend* en Julia, manteniendo la integración del flujo de trabajo a través de un *frontend* en Python.

La búsqueda se ejecuta mediante un algoritmo genético estructurado en múltiples poblaciones independientes o "islas", las cuales evolucionan en paralelo aplicando operadores de mutación, recombinación y simplificación. Periódicamente, las islas intercambian individuos a través de torneos donde la probabilidad de supervivencia depende estrictamente de la bondad de ajuste. Esta topología mitiga la convergencia en mínimos locales y asegura un muestreo exhaustivo del espacio de soluciones.

En el marco de este estudio, el espacio de funciones se genera combinando las variables independientes mediante operadores binarios ($\{+, -, \times, \div, \wedge\}$) y transformaciones unarias ($\{x^{-1}, x^2, e^x, \cos x, \sin x, \ln x\}$). Para garantizar la extracción de relaciones algebraicas interpretables, se impone una restricción explícita de complejidad que limita el número de nodos permitidos en los árboles de expresión.

Durante este proceso de ajuste, PySR construye un frente de Pareto que registra las ecuaciones candidatas evaluadas, contrastando su error frente a su complejidad. Sobre este conjunto, el criterio `model_selection = "best"` aísla el modelo definitivo, seleccionando matemáticamente la expresión que ofrece el compromiso óptimo entre precisión predictiva y parsimonia.

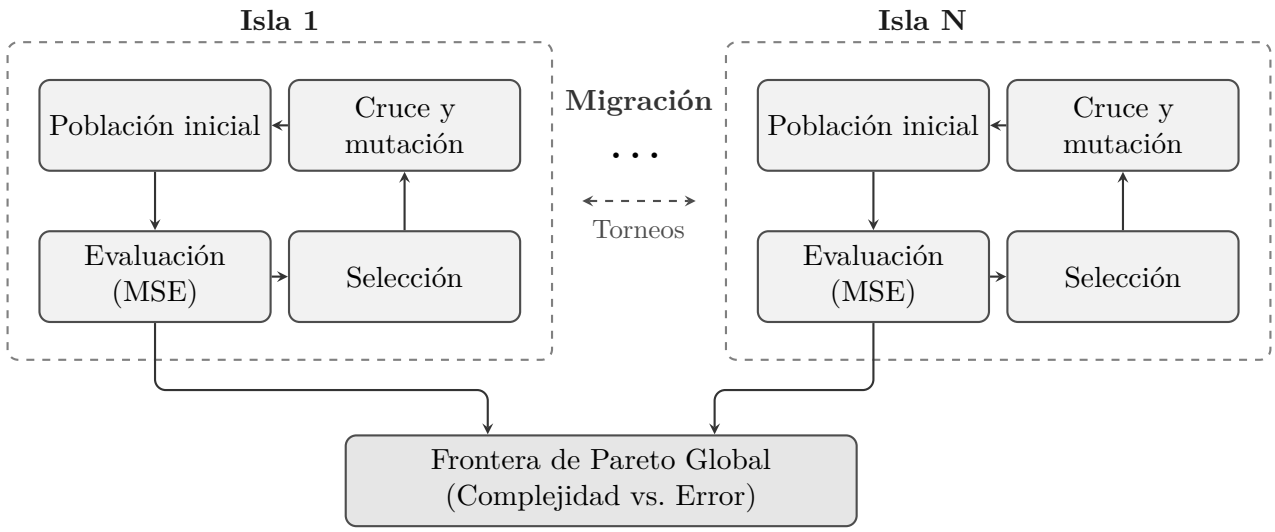

Figura 8: Topología del algoritmo evolutivo en PySR. Múltiples islas evolucionan subpoblaciones en paralelo mediante operaciones de mutación y cruce. Estocásticamente, intercambian individuos (migración por torneos) para evitar convergencia prematura. Todas las islas alimentan un frente de Pareto global que consolida el compromiso óptimo entre parsimonia y precisión predictiva.

5.3.4. QLattice

QLattice, implementado mediante la librería `Feyn` [3], es un motor de regresión simbólica que explora el espacio de modelos (grafos computacionales) mediante un enfoque probabilístico. A diferencia de los métodos evolutivos estándar, genera expresiones matemáticas muestreando una distribución de probabilidad que, partiendo de un estado homogéneo, se actualiza dinámicamente para favorecer las topologías que mejor describen los datos experimentales.

El espacio de búsqueda se define mediante la composición del conjunto de operadores binarios $\{+, \times, e^{-(x^2+y^2)}\}$ y el conjunto de operadores unarios $\{x^2, wx+b, \tanh x, e^{-x^2}, e^x, \ln x, x^{-1}\}$.

Durante el entrenamiento, el algoritmo genera iterativamente poblaciones de grafos candidatos muestreados a partir de la distribución de probabilidad vigente. En la primera etapa, los parámetros continuos de cada grafo se optimizan minimizando la función de pérdida (MSE). Seguidamente, el desempeño de los modelos se evalúa mediante el Criterio de Información Bayesiano (BIC). Las topologías que minimizan este criterio se emplean para actualizar el QLattice, aumentando la probabilidad de generar estructuras algebraicas similares en el siguiente ciclo.

La ventaja fundamental de esta arquitectura es la incorporación inherente del principio de parsimonia. La evolución probabilística del sistema, acoplada a la estricta penalización por complejidad que dicta el BIC, suprime el crecimiento descontrolado de la red que derivaría en sobreajuste. Este mecanismo filtra la adición de términos espurios y fuerza la convergencia hacia modelos compactos que aíslan las relaciones analíticas fundamentales entre las variables.

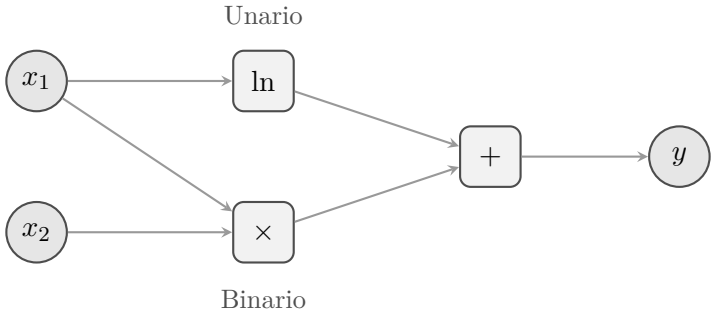

Figura 9: Ejemplo de grafo computacional muestreado por el retículo probabilístico de QLattice para la expresión $y = x_1x_2 + \ln(x_1)$. Los nodos circulares representan los tensores de datos (entradas y predicción), mientras que los nodos rectangulares codifican las operaciones matemáticas.

6. Optimización de Hiperparámetros mediante Optuna

`Optuna` [2] es una biblioteca de código abierto dedicada a la optimización de hiperparámetros en *machine learning*. Destaca por su versatilidad y compatibilidad con la mayoría de *frameworks*, incluidos los discutidos en este trabajo. El proceso requiere definir una función objetivo, a minimizar o maximizar, y el espacio de búsqueda asociado.

El algoritmo de muestreo por defecto es TPE (*Tree-structured Parzen Estimator*), fundamentado en la optimización bayesiana. Sin embargo, en este trabajo se ha optado por el algoritmo `RandomSampler` debido a restricciones computacionales derivadas de la alta dimensionalidad del espacio de hiperparámetros y del reducido número de iteraciones disponibles. En espacios de alta dimensión, TPE requiere un volumen de muestreo considerable para modelar correctamente las densidades de probabilidad subyacentes. Para un número bajo de iteraciones, la sobrecarga analítica de TPE resulta ineficiente. En este régimen, la búsqueda aleatoria pura constituye una línea base (*baseline*) robusta que garantiza una exploración insesgada del espacio, minimizando drásticamente el tiempo de procesamiento por iteración. A continuación, en la Figura 10, se expone un diagrama con el flujo de optimización que lleva a cabo Optuna.

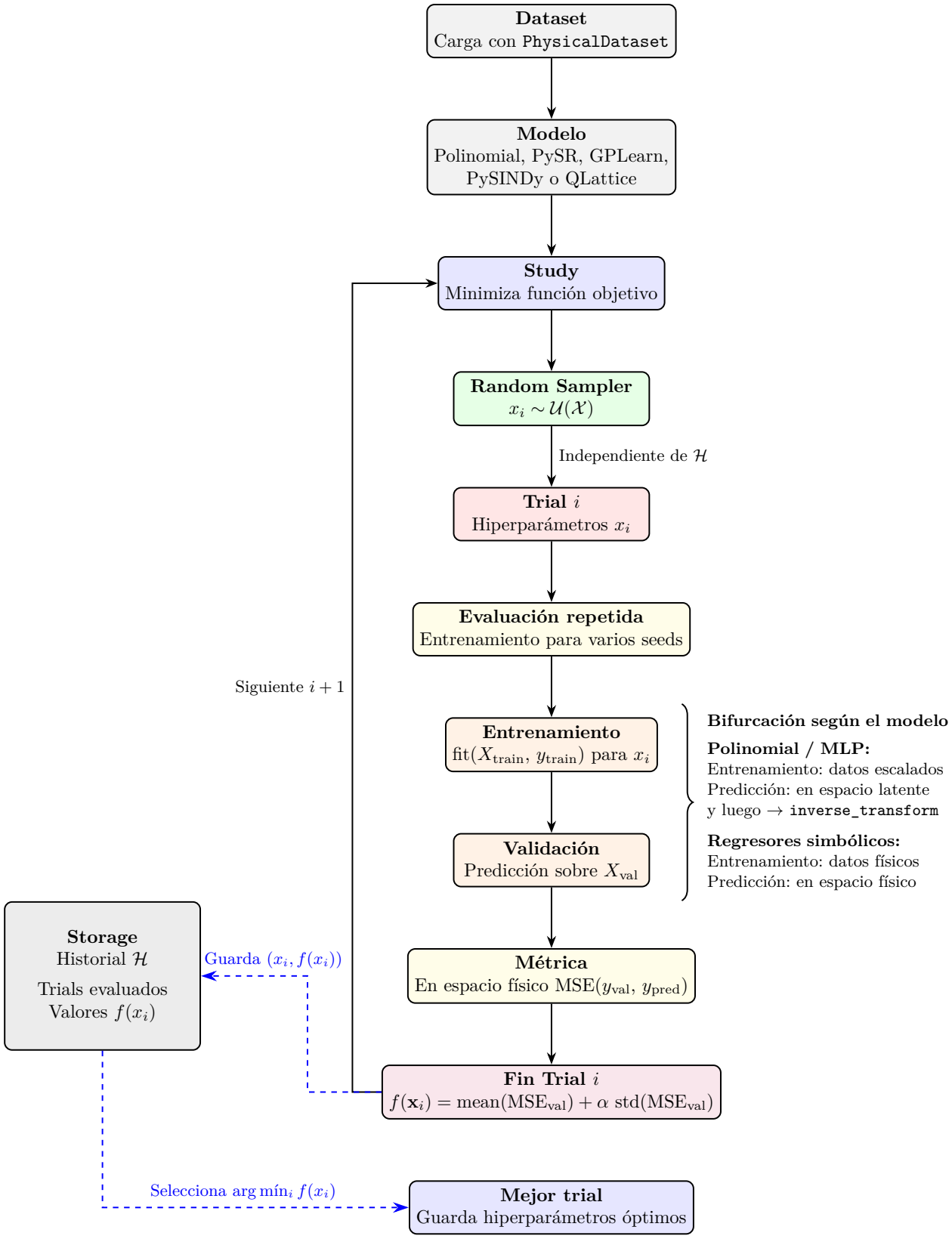

Figura 10: *Pipeline* de optimización de hiperparámetros mediante Optuna.

6.1. Análisis Analítico del Pipeline

Al instanciar `optuna.samplers.RandomSampler`, la asignación x_i en cada iteración sigue una distribución de probabilidad elegida *a priori* (uniforme en este caso) sobre el espacio $\mathcal{X}$, sin extraer información del historial $\mathcal{H}_{i-1}$. Esta independencia entre iteraciones lo convierte en un *baseline* robusto, especialmente adecuado para regímenes de baja dimensionalidad con evaluación fuertemente paralela. Además, al configurarse de forma determinista, se emplea para cada modelo una lista preseleccionada de conjuntos de hiperparámetros común a todos los *datasets*, lo que garantiza una búsqueda homogénea entre experimentos, si bien limita la diversidad de exploración cuando el número de *trials* disponibles es reducido.

La fase de entrenamiento y validación presenta una bifurcación en función del tipo de modelo. La regresión polinómica y las MLP se entrenan en el espacio latente, esto es, con los datos escalados mediante z-score, dado que esta representación mejora la estabilidad numérica y atenúa los problemas asociados a diferencias de escala entre variables. Sus predicciones se obtienen inicialmente en dicho espacio y se transforman posteriormente al espacio físico mediante `inverse_transform` para calcular $f(x_i)$. Los regresores simbólicos, en cambio, operan directamente sobre los datos sin procesar, dado que las expresiones matemáticas que generan no son sensibles a diferencias de escala entre variables.

En cuanto al criterio de optimización, el *pipeline* actual no minimiza únicamente el error individual de cada *trial*, sino una combinación del promedio y la desviación estándar del MSE de validación evaluado sobre distintas semillas de inicialización (regulado por el coeficiente α). Este enfoque reduce la dependencia de resultados exitosos puntuales (óptimos localmente) y favorece la selección de hiperparámetros que promuevan una convergencia estable y reproducible.

Por otro lado, aunque el procedimiento general es idéntico para todos los modelos, los espacios de búsqueda difieren entre sí en función del grado de personalización que permite cada arquitectura: modelos como PySR permiten personalizar un mayor número de hiperparámetros, mientras que otros como QLattice mantienen una parametrización más rígida.

Por último, con el objetivo de establecer una métrica de comparación homogénea de la arquitectura frente a los distintos *datasets*, se adoptó una configuración de hiperparámetros única. Esta se definió heurísticamente como la media aritmética de los valores óptimos hallados individualmente mediante `Optuna` para cada experimento. Aunque el espacio de hiperparámetros no es convexo y este enfoque no garantiza un mínimo global, proporciona un marco de evaluación uniforme que impide el sobreajuste de la configuración a la topología de una ley física específica. No obstante, esta constricción induce inherentemente un rendimiento subóptimo local; una optimización independiente para cada conjunto de datos maximizaría la precisión del modelo, especialmente en los sistemas de mayor complejidad dinámica y composicional.

7. Arquitectura Computacional

El código desarrollado para este estudio está estructurado siguiendo principios de diseño orientado a objetos (OOP), lo que permite alta modularidad y escalabilidad funcional al momento de integrar nuevas arquitecturas de regresión.

7.1. Diseño Orientado a Objetos (Patrón Strategy)

La implementación define una clase abstracta o interfaz común, `BaseModel`, de la que heredan todas las implementaciones específicas. Esto garantiza un contrato polimórfico estricto: el bucle principal de

ejecución no necesita conocer los detalles internos de los regresores simbólicos o perceptrones multicapa, sino únicamente interactuar con sus métodos públicos fundamentales.

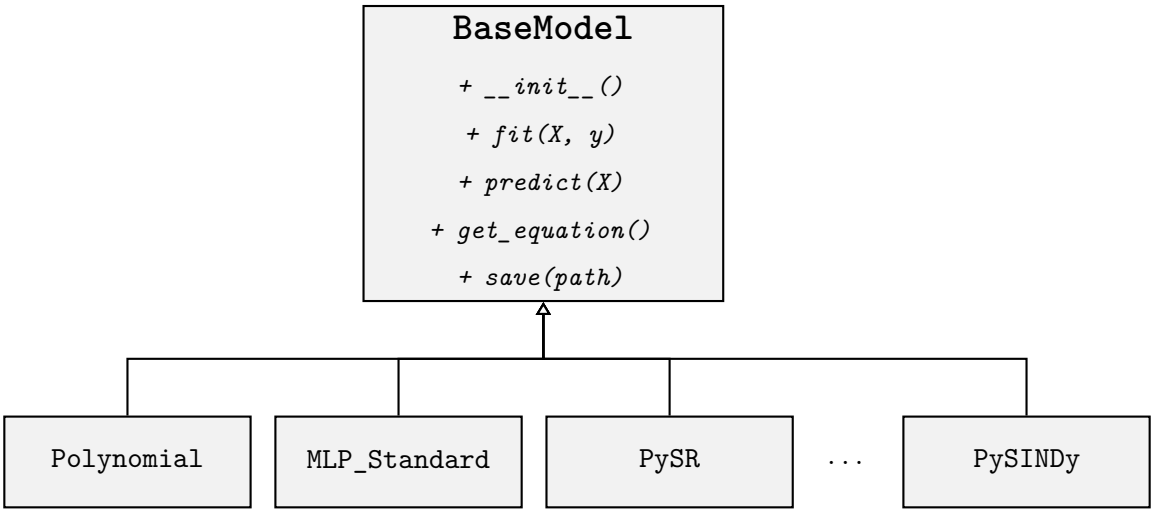

Figura 11: Diagrama de clases: Estructura de herencia de modelos.

7.2. Pipeline de Ejecución

El script `main.py` coordina la ejecución iterando sistemáticamente sobre los conjuntos de datos y el diccionario de modelos instanciados. El flujo de información garantiza que el preprocesamiento (*split* y escalado z-score) se recalcule localmente para cada problema, evadiendo contaminación de datos.

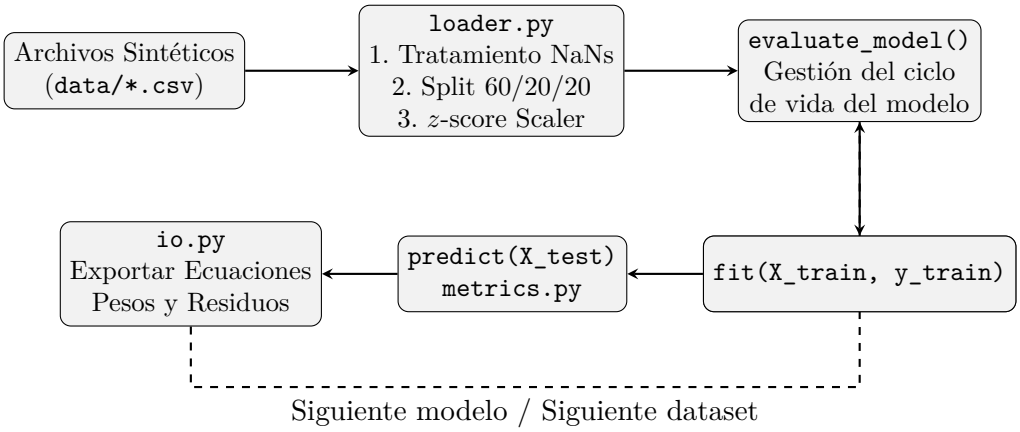

Figura 12: Flujo de tensores e información en el *pipeline* de evaluación.

8. Resultados y Discusión

En esta sección se evalúa el desempeño predictivo de los modelos mediante el Error Cuadrático Medio (MSE) calculados sobre el conjunto de test, tanto dentro como fuera del rango de entrenamiento. Para preservar la relevancia física de los resultados, todas las métricas se reportan en las unidades del espacio físico original.

En las Figuras 13 y 14 se muestran las gráficas correspondientes a las curvas de pérdida (*loss*) para los datos del entrenamiento (*train*) y validación (*validation*), provenientes de la Ley de Coulomb

sin ruido, a lo largo de las épocas de entrenamiento para los modelos MLP estándar y QLattice, respectivamente. La totalidad de las gráficas con las curvas de pérdida para todos los modelos y todas las leyes físicas se encuentra en GitHub [1].

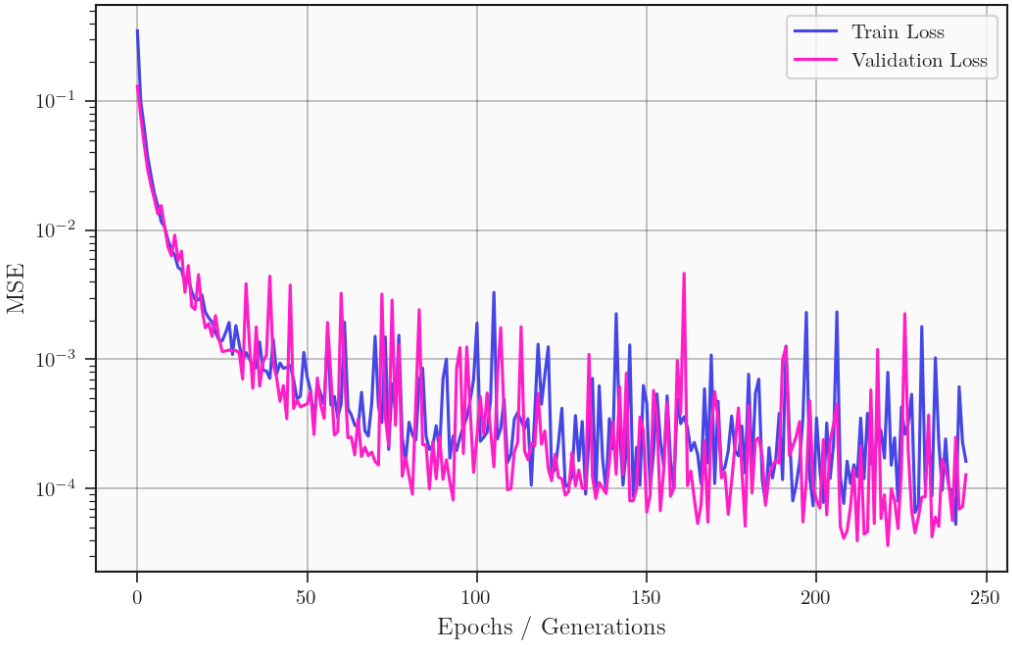

Figura 13: Curvas de pérdidas de entrenamiento (azul) y validación (violeta) para la MLP Estándar sobre el conjunto de datos correspondiente a la Ley de Coulomb sin ruido.

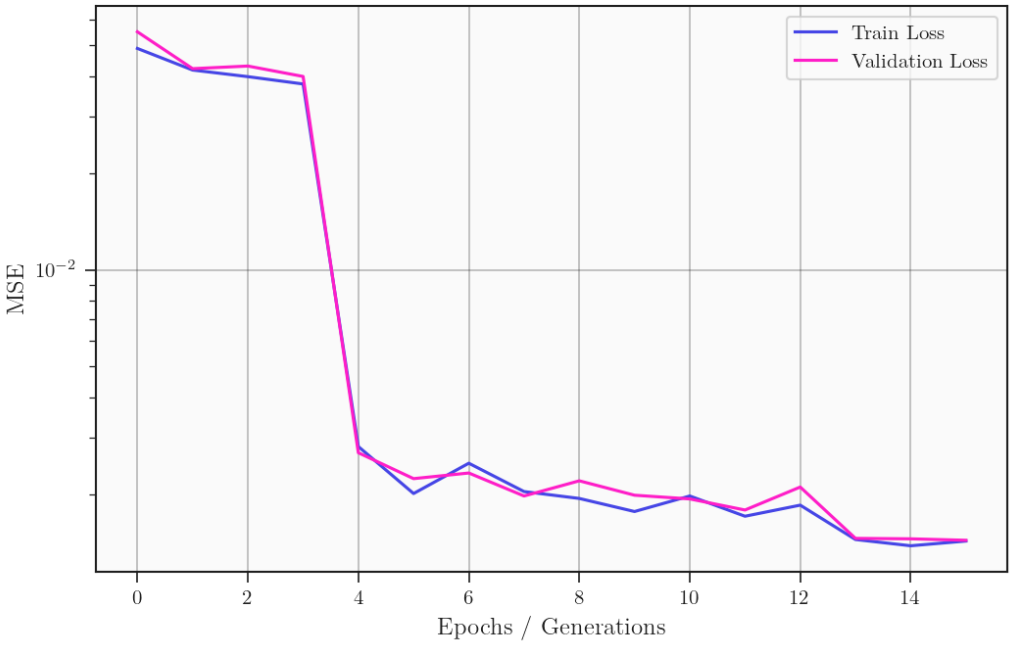

Figura 14: Curvas de pérdidas de entrenamiento (azul) y validación (violeta) para el modelo QLattice sobre el conjunto de datos correspondiente a la Ley de Coulomb sin ruido.

El comportamiento de las curvas de las Figuras 13 y 14 es similar a la mayoría de gráficas almacenadas en GitHub: una caída fuertemente marcada en el valor del MSE durante las primeras iteraciones, para

después presentar un decrecimiento menos acentuado junto con pequeñas fluctuaciones estocásticas. Es especialmente relevante que en ninguna de las gráficas obtenidas se observa un crecimiento de la curva de validación a la vez que decrece la curva de entrenamiento, confirmando que ninguno de los modelos está sobreajustando a los datos de entrenamiento.

En las Tablas 3–11 se resume el desempeño de los modelos ante datos de los sistemas físicos recogidos en la Tabla 1, evaluados tanto dentro del dominio de entrenamiento (*Inside of Domain*, IOD) como fuera de él (*Outside of Domain*, OOD). Para cada nivel de ruido se reporta el error cuadrático medio (MSE) en ambas condiciones, permitiendo contrastar la capacidad de generalización e interpolación de cada enfoque. Cabe mencionar que se emplea notación e (científica) para aumentar y simplificar la legibilidad y comparación.

	Sin Ruido		**Ruido Bajo**		**Ruido Alto**	
Modelo	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**
MLP Estándar	2.03e-4	5.06e-1	3.21e-4	8.84e-1	1.89e-2	2.18e-1
MLP Disperso	7.16e-4	7.94e-1	7.11e-4	9.51e-1	1.87e-2	7.84e-1
MLP Dropout	8.97e-3	9.59e-1	6.70e-3	9.31e-1	4.28e-2	1.04e+0
Polinómico	6.54e-3	5.32e+6	6.64e-3	5.46e+6	2.37e-2	5.18e+6
PySINDy	3.79e-2	4.03e+4	3.81e-2	4.05e+4	5.41e-2	3.59e+4
GPLearn	3.01e-13	0.00e+0	1.76e-4	0.00e+0	1.72e-2	0.00e+0
PySR	3.14e-13	0.00e+0	1.76e-4	0.00e+0	1.72e-2	0.00e+0
QLattice	1.42e-3	3.96e-1	2.72e-4	1.67e-6	1.80e-2	3.03e-3

Tabla 3: Resultados para la **Ley de Coulomb** ($F = q_1 q_2 / r^2$).

	Sin Ruido		**Ruido Bajo**		**Ruido Alto**	
Modelo	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**
MLP Estándar	4.09e-9	1.19e-3	1.60e-4	7.57e-3	1.41e-2	1.73e-4
MLP Disperso	2.23e-5	3.08e+0	1.99e-4	3.14e+0	1.42e-2	5.89e+0
MLP Dropout	1.36e-3	3.79e-2	3.45e-3	1.11e-1	1.70e-2	5.90e-2
Polinómico	8.37e-14	5.01e-7	1.42e-4	9.37e+0	1.41e-2	2.03e+2
PySINDy	1.92e-14	3.01e-13	1.42e-4	2.95e-9	1.40e-2	1.27e-5
GPLearn	3.51e-5	5.30e-4	1.81e-4	5.30e-4	1.41e-2	5.30e-4
PySR	0.00e+0	0.00e+0	1.42e-4	6.30e-8	1.41e-2	3.65e-6
QLattice	5.37e-15	N/D	1.45e-4	2.36e-6	1.43e-2	7.47e-4

Tabla 4: Resultados para el **Oscilador Armónico** ($F = -x$).

	Sin Ruido		**Ruido Bajo**		**Ruido Alto**	
Modelo	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**	**MSE$_{\text{IOD}}$**	**MSE$_{\text{OOD}}$**
MLP Estándar	2.01e-6	1.00e+2	2.48e-4	9.33e+1	2.45e-2	9.10e+1
MLP Disperso	3.17e-5	3.03e+2	2.66e-4	3.03e+2	2.29e-2	3.23e+2
MLP Dropout	3.74e-3	1.36e+2	1.59e-3	1.09e+2	2.63e-2	1.30e+2
Polinómico	3.45e-7	6.30e+4	2.32e-4	9.23e+4	2.23e-2	8.38e+4
PySINDy	2.20e-5	4.95e+1	6.22e-4	1.76e+1	2.24e-2	5.50e+1
GPLearn	3.41e-13	1.20e-30	2.32e-4	1.20e-30	2.24e-2	1.20e-30
PySR	4.04e-11	1.32e-8	2.32e-4	4.12e-6	2.24e-2	5.90e-4
QLattice	1.80e-7	3.34e-5	2.38e-4	5.31e-3	2.26e-2	9.66e-3

Tabla 5: Resultados para la **Tercera Ley de Kepler** ($T = r^{3/2}$).

	Sin Ruido		Ruido Bajo		Ruido Alto	
Modelo	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**
MLP Estándar	1.81e-5	9.21e+0	1.13e-4	7.91e+0	1.01e-2	8.73e+0
MLP Disperso	9.73e-5	1.01e+1	2.12e-4	1.02e+1	1.03e-2	1.10e+1
MLP Dropout	2.25e-3	1.23e+1	3.05e-3	1.06e+1	1.59e-2	1.64e+1
Polinómico	3.15e-5	3.29e+3	1.27e-4	2.85e+3	9.83e-3	1.69e+3
PySINDy	1.76e-3	8.91e+3	1.69e-3	6.25e+3	1.15e-2	3.39e+3
GPLearn	3.21e-13	0.00e+0	9.35e-5	0.00e+0	9.73e-3	0.00e+0
PySR	3.21e-13	0.00e+0	9.35e-5	0.00e+0	9.73e-3	0.00e+0
QLattice	1.24e-8	6.09e-7	3.42e-4	1.47e+0	1.02e-2	2.24e+1

Tabla 6: Resultados para la **Ley de los Gases Ideales** ($P = nT/V$).

	Sin Ruido		Ruido Bajo		Ruido Alto	
Modelo	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**
MLP Estándar	5.52e-5	1.24e+3	6.15e-4	1.33e+3	5.00e-2	1.42e+3
MLP Disperso	2.79e-4	1.57e+3	7.81e-4	1.60e+3	4.89e-2	1.62e+3
MLP Dropout	1.43e-2	1.61e+3	1.33e-2	1.59e+3	6.31e-2	1.42e+3
Polinómico	2.41e-5	1.21e+3	5.00e-4	2.02e+3	4.81e-2	1.71e+3
PySINDy	9.44e-4	3.21e-1	1.43e-3	3.28e-1	5.04e-2	1.16e+1
GPLearn	4.41e-1	N/D	4.41e-1	N/D	4.92e-1	N/D
PySR	2.14e-12	0.00e+0	4.71e-4	0.00e+0	4.79e-2	1.87e-3
QLattice	3.97e-4	1.22e+3	2.45e-3	1.70e+3	4.98e-2	3.93e-1

Tabla 7: Resultados para el **Alcance del Proyectil** ($R = v_0^2 \sin(2\theta)$).

	Sin Ruido		Ruido Bajo		Ruido Alto	
Modelo	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**
MLP Estándar	3.00e-5	7.91e+2	1.15e-4	8.05e+2	7.11e-3	7.83e+2
MLP Disperso	5.76e-5	8.13e+2	1.30e-4	8.14e+2	7.04e-3	8.39e+2
MLP Dropout	2.22e-3	7.90e+2	3.17e-3	7.95e+2	9.49e-3	7.84e+2
Polinómico	1.62e-3	5.83e+2	1.70e-3	5.41e+2	8.38e-3	3.46e+3
PySINDy	1.01e-2	2.62e+2	1.02e-2	2.63e+2	1.72e-2	2.61e+2
GPLearn	1.64e-1	5.78e+2	1.64e-1	5.78e+2	1.71e-1	5.78e+2
PySR	1.48e-4	2.34e+2	1.25e-4	N/D	8.92e-3	2.85e+2
QLattice	1.52e-4	1.92e+2	2.04e-4	1.89e+2	6.88e-3	1.92e+2

Tabla 8: Resultados para la **Dilatación del Tiempo** ($t' = t/\sqrt{1 - v^2}$).

	Sin Ruido		Ruido Bajo		Ruido Alto	
Modelo	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**
MLP Estándar	9.74e-7	4.84e+0	1.19e-5	3.15e+0	9.12e-4	4.64e+0
MLP Disperso	1.70e-5	1.58e+0	1.61e-5	1.54e+0	8.88e-4	1.56e+0
MLP Dropout	3.12e-4	4.87e+0	1.48e-4	4.12e+0	1.05e-3	3.96e+0
Polinómico	9.21e-6	4.88e+3	1.78e-5	5.34e+3	8.77e-4	5.09e+3
PySINDy	1.97e-4	5.10e+2	2.05e-4	5.14e+2	1.06e-3	5.30e+2
GPLearn	5.85e-2	2.87e-1	5.85e-2	2.87e-1	5.92e-2	2.87e-1
PySR	4.70e-15	0.00e+0	8.55e-6	0.00e+0	8.60e-4	0.00e+0
QLattice	1.48e-12	1.78e-13	8.70e-6	7.76e-8	8.61e-4	3.45e-7

Tabla 9: Resultados para la **Desintegración Radiactiva** ($N = e^{-\lambda t}$).

	Sin Ruido		Ruido Bajo		Ruido Alto	
Modelo	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**
MLP Estándar	2.93e-7	1.28e+1	1.61e-5	7.39e+0	9.72e-4	1.02e+1
MLP Disperso	9.82e-6	4.86e+0	1.99e-5	4.95e+0	9.83e-4	4.99e+0
MLP Dropout	1.41e-4	8.92e+0	1.38e-4	8.68e+0	1.36e-3	9.78e+0
Polinómico	9.55e-6	3.14e+3	1.83e-5	2.50e+3	9.55e-4	5.45e+3
PySINDy	2.01e-4	2.65e+2	2.07e-4	2.67e+2	1.18e-3	5.61e+2
GPLearn	9.13e-2	1.34e-1	9.13e-2	1.34e-1	9.21e-2	1.34e-1
PySR	8.78e-14	0.00e+0	8.76e-6	1.46e-8	9.44e-4	7.29e-12
QLattice	6.22e-10	1.81e-16	9.27e-6	4.71e-16	9.43e-4	4.90e-7

Tabla 10: Resultados para la **Ley del Enfriamiento de Newton** ($T = 1 + e^{-kt}$).

	Sin Ruido		Ruido Bajo		Ruido Alto	
Modelo	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**	**MSE_{IOD}**	**MSE_{OOD}**
MLP Estándar	9.67e-7	2.71e+3	5.47e-5	4.17e+2	3.53e-3	4.18e+3
MLP Disperso	9.19e-6	3.02e+0	4.60e-5	1.38e+0	3.52e-3	1.56e+0
MLP Dropout	3.68e-4	6.65e+2	2.63e-4	9.09e+2	3.88e-3	8.20e+2
Polinómico	1.49e-5	1.41e+17	4.96e-5	1.40e+17	3.53e-3	1.11e+17
PySINDy	8.55e-5	1.17e+1	1.23e-4	1.16e+1	1.47e-2	1.24e+2
GPLearn	7.95e-14	0.00e+0	3.48e-5	0.00e+0	3.50e-3	0.00e+0
PySR	7.95e-14	0.00e+0	3.48e-5	0.00e+0	3.50e-3	0.00e+0
QLattice	1.08e-12	1.25e-8	3.56e-5	5.29e-6	3.52e-3	2.66e-4

Tabla 11: Resultados para la **Entropía de Boltzmann** ($S = \ln \Omega$).

De manera general, se observa que en el rango de estudio, IOD, los modelos presentan MSE relativamente bajos, pero este hecho carece de relevancia física si los resultados no pueden extrapolarse a un dominio no estudiado. Por ello, el análisis de la calidad de las ecuaciones ofrecidas por los regresores y el comportamiento de las MLP se ponen a prueba en el dominio OOD. En este contexto, se produce una clara dominancia de los regresores simbólicos, particularmente de PySR, GPLearn y QLattice, los cuales promedian un MSE fuera del dominio de entrenamiento muy inferior respecto al resto de modelos.

Estos resultados se muestran de manera más visual a continuación. En las Figuras 15 y 16 se presentan gráficos de barras con el MSE para cada modelo y nivel de ruido, relativas a los datos de la Ley de Kepler y el oscilador armónico, para muestras fuera del rango de entrenamiento. La bondad de la predicción se clasifica según el criterio establecido en la Sección 4.2: exacta, precisa, aproximada e incorrecta. Asimismo, el resto de gráficos de barras correspondientes al resto de leyes físicas se encuentran en GitHub [1].

Si bien estos gráficos de barras permiten un análisis detallado, es complejo dilucidar una conclusión clara de manera general. Por ello, se condensa la información en la Figura 17, obviando los valores particulares del MSE para cada modelo y mostrando exclusivamente la clasificación por categoría.

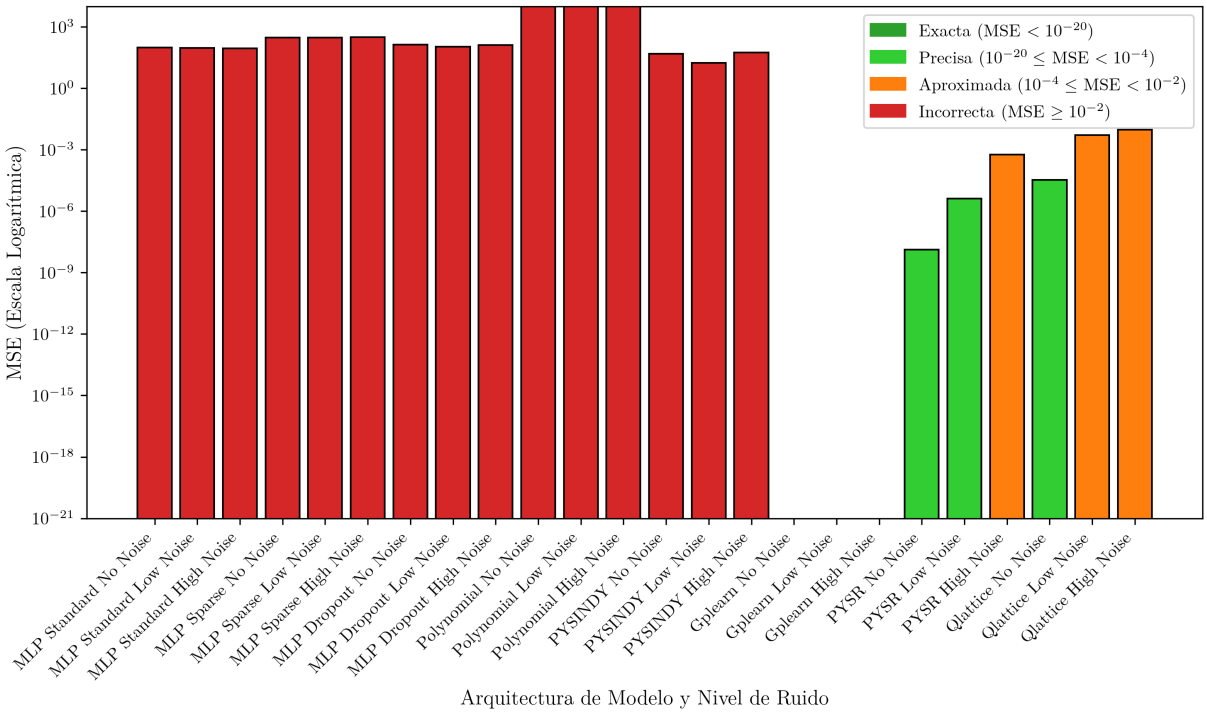

Figura 15: Gráfico de barras con el MSE para cada modelo y nivel de ruido con código de color para la bondad de la predicción (verde oscuro: exacta, verde: precisa, naranja: aproximada, rojo: incorrecta) relativa a los datos de la **Tercera Ley de Kepler** para muestras fuera del rango de entrenamiento (OOD).

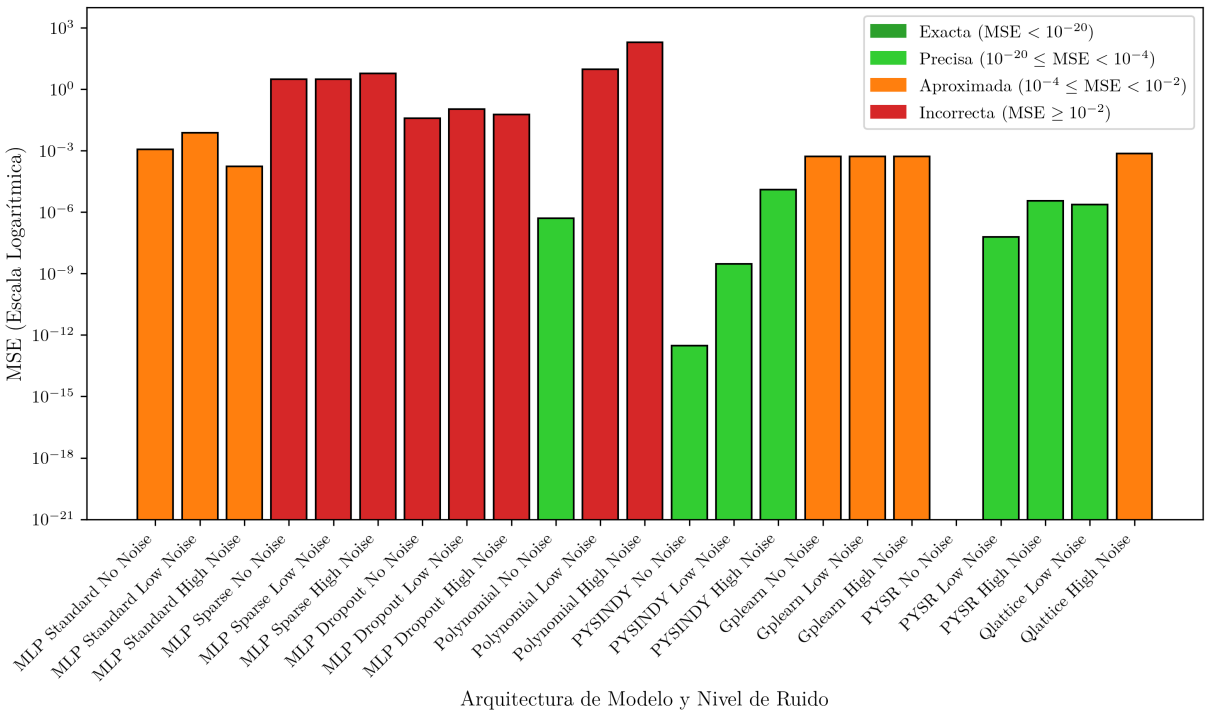

Figura 16: Gráfico de barras con el MSE para cada modelo y nivel de ruido con código de color para la bondad de la predicción (verde oscuro: exacta, verde: precisa, naranja: aproximada, rojo: incorrecta) relativa a los datos del **Oscilador Armónico** para muestras fuera del rango de entrenamiento (OOD).

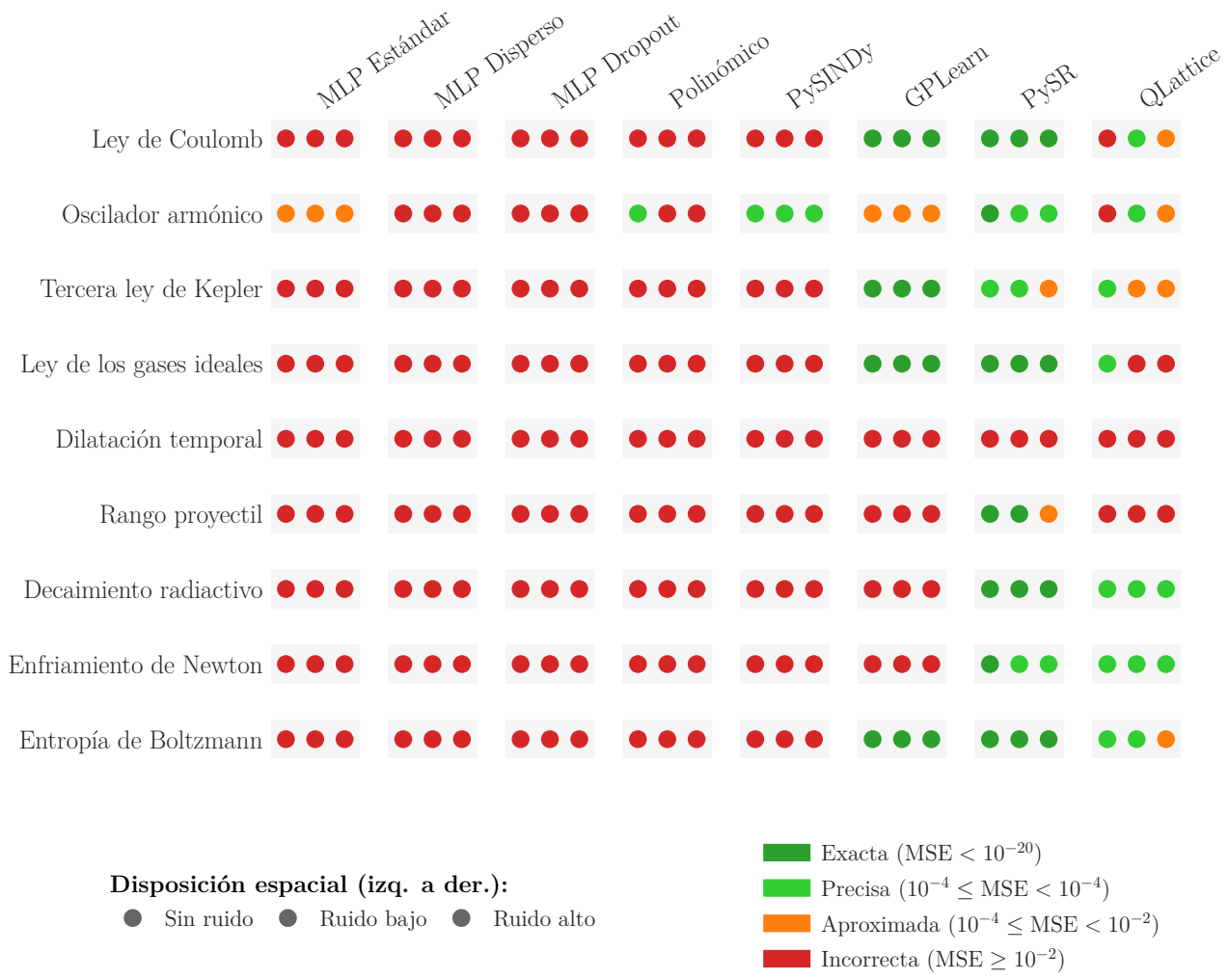

Figura 17: Bondad de la predicción de cada modelo para cada nivel de ruido y cada ley para muestras fuera del rango de entrenamiento (OOD).

Finalmente, la Figura 17 se desglosa en tres tablas en las Figuras 18–20, mostrando no solo la categoría según el MSE, sino también las leyes obtenidas por los regresores. Así se refleja la verdadera ventaja de los regresores simbólicos, los cuales no solo logran unos mejores resultados en términos de MSE, sino que producen expresiones físicamente interpretables.

		Redes Neuronales				Regresores Simbólicos			
Ley Física	Forma Analítica	Estándar	Dispersa	Dropout	Polinomial	PySINDy	GPLearn	PySR	QLattice
Ley de Coulomb	q_1q_2/r^2	–	–	–	$-0.0382q_1^5 - 0.07292q\ldots$	$-1.037 + 6.685 \times 10^{-1}\ldots$	q_1q_2/r^2	$q_1q_2/(r^2)$	$2.042(-0.3664 + 0.9505/(0.4956\ldots$
Oscilador armónico	$-x$	–	–	–	$2.52 \times 10^{-8}x^5 - 1.3\ldots$	$-x$	$-0.995x$	$-x - x + x$	$-7.376(1.356x + 0.002145)\ldots$
Tercera ley de Kepler	$r^{3/2}$	–	–	–	$-0.005556r^5 + 0.05599r^4\ldots$	$-5.211 \times 10^{-2} + 5.788\ldots$	$r^{3/2}$	$r^{1.4999973}$	$-1.481(-0.5112r - 0.008606)\ldots$
Ley de los gases ideales	nT/V	–	–	–	$0.009562T^5 - 0.0006943T^4\ldots$	$-5.103 \times 10^{-3} + 5.501\ldots$	Tn/V	Tn/V	$-0.7828(4.93T + 0.0003735)(\ldots$
Dilatación temporal	$t/\sqrt{1-v^2}$	–	–	–	$-0.01782t^5 + 0.2808t^4v\ldots$	$2.415 \times 10^{-1} + 5.793\ldots$	$t(v+1)$	$t(1.6901535 + 1/\cos(v^{1.3647355}\ldots$	$-2.098(1.039 - 0.7927v)(2.13\ldots$
Rango proyectil	$v_0^2\sin(2\theta)$	–	–	–	$-0.006502\theta^5 + 2.152\ldots$	$-6.142 \times 10^{-1} + 1.266\ldots$	$v_0^{3/2}\sqrt{\log(\theta}\ldots$	$v_0^2\sin(\theta + \theta)$	$-15.61\exp(-2.0(0.142 - 0.12\ldots$
Decaimiento radiactivo	$e^{-\lambda t}$	–	–	–	$-0.0279\lambda^5 + 0.07706\ldots$	$9.853 \times 10^{-1} + 1.136\ldots$	$\cos(\sqrt{t})$	$\exp(\lambda)^{-t}$	$\exp((1.303 \times 10^{-6} - 0.\ldots$
Enfriamiento de Newton	$1 + e^{-kt}$	–	–	–	$-0.03169k^5 + 0.0738k^4t\ldots$	$2.002 + 1.984 \times 10^{-1}\ldots$	1.3661202185792 3	$\exp(-kt) + 1.0$	$\exp((1.272 \times 10^{-5} - 0.\ldots$
Entropía de Boltzmann	$\ln(\Omega)$	–	–	–	$0.00011729^5 - 0.003861\ldots$	$1.737 + 8.828 \times 10^{-2}\ldots$	$\log(\Omega)$	$\log(\Omega)$	$\log(0.408\Omega - 9.859\ldots$

Exacta (MSE < 10^{-20}) Precisa ($10^{-20} \leq$ MSE < 10^{-4}) Aproximada ($10^{-4} \leq$ MSE < 10^{-2}) Incorrecta (MSE $\geq 10^{-2}$)

Figura 18: Tabla comparativa de las expresiones matemáticas recuperadas por los regresores simbólicos y el MSE_{OOD} para cada modelo para datos sin ruido.

		Redes Neuronales				Regresores Simbólicos			
Ley Física	Forma Analítica	Estándar	Dispersa	Dropout	Polinomial	PySINDy	GPLearn	PySR	QLattice
Ley de Coulomb	q_1q_2/r^2	–	–	–	$-0.03695q_1^5 - 0.07018q\ldots$	$-1.036 + 6.672 \times 10^{-1}\ldots$	q_1q_2/r^2	$q_1q_2/(r^2)$	$2.086(0.3505q_1 + 0.001896)\ldots$
Oscilador armónico	$-x$	–	–	–	$0.0001097x^5 + 9.195\ldots$	$-x$	$-0.995x$	$-x - 0.00025102496$	$-0.9994x - 0.001266$
Tercera ley de Kepler	$r^{3/2}$	–	–	–	$-0.006601r^5 + 0.06382r^4\ldots$	$-1.259 \times 10^{-1} + 8.277\ldots$	$r^{3/2}$	$r^{1.5000477}$	$2.811(0.4257r - 0.07997)\ldots$
Ley de los gases ideales	nT/V	–	–	–	$0.03444T^5 - 0.01838T^4V\ldots$	$-9.143 \times 10^{-3} + 3.459\ldots$	Tn/V	Tn/V	$-2.466(0.003451 - 0.1432T)(0\ldots$
Dilatación temporal	$t/\sqrt{1-v^2}$	–	–	–	$-0.01266t^5 + 0.2917t^4v\ldots$	$2.416 \times 10^{-1} + 5.745\ldots$	$t(v+1)$	$t/(\cos(v1.5881233)^{0.4212728})$	$0.7705t + 0.002551 - 0.6356(\ldots$
Rango proyectil	$v_0^2\sin(2\theta)$	–	–	–	$0.0005253\theta^5 + 2.203\ldots$	$-6.159 \times 10^{-1} + 1.256\ldots$	$v_0^{3/2}\sqrt{\log(\theta}\ldots$	$v_0^2\sin(\theta+\theta)$	$17.86\exp(-2.0(0.328v_0\ldots$
Decaimiento radiactivo	$e^{-\lambda t}$	–	–	–	$-0.02738\lambda^5 + 0.07748\ldots$	$9.856 \times 10^{-1} + 1.13\ldots$	$\cos(\sqrt{t})$	$\exp(t)^{-\lambda}$	$0.9995\exp((0.0008908 - 0.5667\ldots$
Enfriamiento de Newton	$1 + e^{-kt}$	–	–	–	$-0.0308k^5 + 0.07308k^4t\ldots$	$2.002 + 1.995 \times 10^{-1}\ldots$	1.36612021857923	$\exp(k(-1.0004783))^t + 1.000121$	$\exp((0.000273 - 0.5565t)(1.7\ldots$
Entropía de Boltzmann	$\ln(\Omega)$	–	–	–	$0.00011660\Omega^5 - 0.003842\ldots$	$1.738 + 8.819 \times 10^{-2}\ldots$	$\log(\Omega)$	$\log(\Omega)$	$1.001\log(0.2768\Omega - 0.00\ldots$

Exacta (MSE $< 10^{-20}$) Precisa ($10^{-20} \leq$ MSE $< 10^{-4}$) Aproximada ($10^{-4} \leq$ MSE $< 10^{-2}$) Incorrecta (MSE $\geq 10^{-2}$)

Figura 19: Tabla comparativa de las expresiones matemáticas recuperadas por los regresores simbólicos y el MSE_{OOD} para cada modelo para datos con ruido bajo.

		Redes Neuronales				Regresores Simbólicos			
Ley Física	Forma Analítica	Estándar	Dispersa	Dropout	Polinomial	PySINDy	GPLearn	PySR	QLattice
Ley de Coulomb	q_1q_2/r^2	–	–	–	$-0.04889q_1^5 - 0.09459q\ldots$	$-9.682 \times 10^{-1} + 2.919\ldots$	q_1q_2/r^2	$q_1q_2/(r^2)$	$2.861(0.8354q_1 + 0.002987)\ldots$
Oscilador armónico	$-x$	–	–	–	$-0.0006365x^5 + 0.00138x^4\ldots$	$-9.992 \times 10^{-1}x$	$-0.995x$	$-x + 0.0019095138$	$-0.9964x + 0.01087$
Tercera ley de Kepler	$r^{3/2}$	–	–	–	$-0.006829r^5 + 0.07283r^4\ldots$	$-5.532 \times 10^{-2} + 5.813\ldots$	$r^{3/2}$	$r^{1.5005707}$	$2.328\sqrt{0.2723r - 0.02623}\ldots$
Ley de los gases ideales	nT/V	–	–	–	$0.04708T^5 + 0.04781T^4V\ldots$	$4.225 \times 10^{-2} + 4.059\ldots$	Tn/V	Tn/V	$-1.513(0.4255V - 2.057)(4.18\ldots$
Dilatación temporal	$t/\sqrt{1-v^2}$	–	–	–	$-0.17t^5 + 0.4091t^4v\ldots$	$2.438 \times 10^{-1} + 5.981\ldots$	$t(v+1)$	$t/\cos((v + 0.16311686)^2)$	$0.7697t - 0.8242(0.901t - 0.\ldots$
Rango proyectil	$v_0^2\sin(2\theta)$	–	–	–	$-1.153\theta^5 + 2.237\ldots$	$-7.959 \times 10^{-2} - 1.209\ldots$	$v_0^{3/2}\sqrt{\log(\theta}\ldots$	$v_0^2\sin(\theta 2.001247)$	$1.25(0.6304\theta + 0.1075)\ldots$
Decaimiento radiactivo	$e^{-\lambda t}$	–	–	–	$-0.03926\lambda^5 + 0.07949\ldots$	$9.868 \times 10^{-1} + 1.124\ldots$	$\cos(\sqrt{t})$	$\exp(\lambda)^{-t}$	$1.003\exp((1.602\lambda + 0.0\ldots$
Enfriamiento de Newton	$1 + e^{-kt}$	–	–	–	$-0.02462k^5 + 0.06923k^4\ldots$	$1.977 + 1.685 \times 10^{-1}\ldots$	1.36612021857923	$\exp(-kt) + 1.0000027$	$1.003\exp((0.8531k + 0.003694\ldots$
Entropía de Boltzmann	$\ln(\Omega)$	–	–	–	$0.00010460\Omega^5 - 0.003543\ldots$	$2.313 \times 10^{-1}\Omega + 2\ldots$	$\log(\Omega)$	$\log(\Omega)$	$1.006\log(0.2613\Omega + 0.00\ldots$

Exacta (MSE $< 10^{-20}$) Precisa ($10^{-20} \leq$ MSE $< 10^{-4}$) Aproximada ($10^{-4} \leq$ MSE $< 10^{-2}$) Incorrecta (MSE $\geq 10^{-2}$)

Figura 20: Tabla comparativa de las expresiones matemáticas recuperadas por los regresores simbólicos y el MSE_{OOD} para cada modelo para datos con ruido alto.

De la figura 17 en adelante, se observa una clara superioridad de los regresores simbólicos PySR, GPLearn y QLattice sobre el resto de modelos. La razón de este hecho es que son capaces de generar relaciones matemáticas sencillas con diferentes operadores, al contrario que las MLP, que no permiten proporcionar expresión matemática alguna, y no únicamente polinomios, como PySINDy y el regresor polinomial. Además, al contar con limitaciones de complejidad intrínsecas en sus funciones de pérdida, el espacio de soluciones se restringe a expresiones compactas y evita el sobreajuste a patrones locales del conjunto de entrenamiento. Así, especialmente GPLearn y PySR respetan el principio de parsimonia, particularmente relevante en casos con ruido, y extrapolan su extraordinario rendimiento en datos IOD a los datos OOD.

Antes de entrar en la discusión detallada de los resultados, es necesario analizar cómo el espacio de operadores disponible en cada algoritmo determina su rendimiento. Aunque excluir funciones no lineales simplificaría el análisis, reduciría drásticamente el alcance físico del estudio, ya que la mayoría de las leyes de la naturaleza involucran relaciones sinusoidales, exponenciales o logarítmicas. Por ello, se han definido previamente los operadores permitidos en cada modelo. Esta restricción estructural se traduce en la incapacidad de GPLearn para reproducir leyes exponenciales y de QLattice para identificar relaciones sinusoidales. Por su parte, los algoritmos de regresión restantes (Regresor Polinomial y PySINDy) presentan una limitación inductiva fundamental: parten de la premisa de que es posible recuperar relaciones matemáticas complejas mediante combinaciones de operaciones simples, lo que genera un sesgo significativo al modelar dinámicas no lineales.

Respecto a las categorías de las Figuras 17–20, en primer lugar, la clasificación de “exacta” representa casos de éxito en la predicción, observándose exclusivamente en los regresores, muestra de que han

logrado obtener las expresiones subyacentes de los datos sintéticos. El modelo que cuenta con más fórmulas exactas es PySR, especialmente en casos sin ruido, aunque consigue reproducir exactamente 4 ecuaciones en caso de ruido alto. Por otro lado, GPLearn también consigue recuperar 4 ecuaciones (curiosamente, el oscilador armónico no es una de ellas), demostrando su superioridad frente al resto de modelos.

En segundo lugar, cuando la predicción se denota como "precisa", de nuevo solo logrado por los regresores simbólicos, sugiere que las ecuaciones difieren de la ley física que buscan modelar únicamente en el valor de las constantes. Esto ocurre ocasionalmente en PySR, especialmente en casos con ruido, y en QLattice, el cual parece captar el comportamiento fundamental de las leyes pero sus resultados poseen una complejidad muy alta (gran número de ecuaciones "precisas", pero ningún resultado "exacto"), lo que produce aumentos rápidos del MSE. La mayor complejidad en las ecuaciones de QLattice se debe a su arquitectura probabilística, la cual requiere de un gran número de iteraciones para estabilizarse completamente. A este hecho hay que sumarle que el criterio BIC penaliza la complejidad en función del número de parámetros libres, una métrica que no captura adecuadamente la complejidad topológica de la expresión resultante.

En tercer lugar, la clasificación de "aproximada" indica un entendimiento insuficiente de la física subyacente. En particular, cuando una expresión de un regresador simbólico cae en esta categoría, manifiesta que se ha conseguido una fórmula que produce datos cercanos a los reales, pero la forma funcional no es adecuada o contiene constantes que modifican altamente las predicciones. Este tipo de expresiones aparecen especialmente en las obtenidas a partir de datos sintéticos con mucho ruido, para los cuales los modelos pueden tratar de sobreajustar sus términos para obtener mejores MSE en el entrenamiento, resultando en un deterioro en su adecuación general debido a su elevada complejidad.

Por último, si no se ha logrado una predicción con un error relativo medio inferior al 10 %, esta se clasifica como "incorrecta". En esta clasificación entran la mayoría de los modelos de perceptrón multicapa, incapaces de aproximar la física para muestras de datos fuera del rango de entrenamiento, independientemente del nivel de ruido, así como la mayoría de predicciones dadas por los modelos de regresión polinómica y PySINDy. La presencia de los regresores simbólicos en esta categoría es causa de la insuficiencia de los términos polinómicos para capturar relaciones multivariables no triviales; sí sirven, en cambio, para casos sencillos.

Adicionalmente, cabe señalar que ninguno de los modelos consigue reproducir la ecuación de dilatación temporal. Esto ocurre debido a que una gran parte de los puntos estudiados caen en la zona donde $t' \approx t$, y que la ecuación subyacente tiene una alta complejidad, por lo que los modelos no consiguen identificarla. Probablemente, con un mejor condicionamiento de los datos, esto es, para valores de velocidad cercanos a la unidad, los modelos más avanzados serían capaces de extraer la expresión analítica real.

Respecto al análisis de la recuperación de ecuaciones en concreto (Figuras 18–20), encontramos varios sucesos dignos de comentar. El primero lo encontramos con la expresión del oscilador armónico, la ecuación más sencilla de todas, que es capaz de ser recuperada por todos los modelos de regresión simbólica salvo por QLattice, no logra converger hacia la solución correcta en el caso sin ruido. Lógicamente, aquellas expresiones con términos exponenciales solo pudieron ser resueltas por los modelos que contenían en su base de operadores la exponencial, a saber: PySR y QLattice. Sin embargo, en cuanto a la expresión con operadores sinusoidales (rango de proyectil), presentes en la base de operadores de PySR y GPLearn, solo es obtenida de manera exacta por el primero de ellos. El último caso reseñable se trata de la fórmula de la entropía de Boltzmann, para la cual, tanto PySR como GPLearn son capaces de recuperar la ecuación exacta, mientras que QLattice obtiene la forma logarítmica pero no su argumento, no obstante, como el logaritmo crece lentamente, su MSE no se dispara al evaluarlo en un rango no muy alejado del original.

Finalmente, evaluamos la robustez de los modelos frente a distintos niveles de ruido. La introducción de ruido leve reduce la convergencia hacia expresiones analíticas exactas. De forma antiintuitiva, esta perturbación beneficia la búsqueda topológica en QLattice, que logra recuperar la forma funcional de la Ley de Coulomb y del oscilador armónico; sin embargo, sobreajusta la ecuación de los gases ideales al introducir constantes no físicas, lo que dispara el error cuadrático medio.

Al transicionar a ruido alto, PySR y QLattice sufren una degradación esperada: logran conservar las relaciones algebraicas subyacentes, pero desajustan las constantes de acoplamiento. El resto de modelos resulta insensible a este aumento de ruido dada la deficiencia basal de sus predicciones.

En definitiva: el ruido penaliza fuertemente la precisión paramétrica. No obstante, un algoritmo capaz de deducir la expresión correcta con datos carentes de ruido tiende a preservar dicha estructura analítica frente a perturbaciones estocásticas, limitándose a absorber la varianza en los parámetros libres de la ecuación.

9. Conclusiones

En este estudio se ha evaluado la capacidad de inferencia analítica de arquitecturas de perceptrón multicapa (MLP), modelos polinómicos y regresores simbólicos (PySR, QLattice, GPLearn, PySINDy) frente a conjuntos de datos de complejidad composicional creciente y perturbación estocástica variable. Para garantizar la validez del trabajo, se ha asegurado una estricta separación en las particiones de datos, y se ha explorado el espacio de hiperparámetros estocásticamente mediante Optuna.

El análisis del rendimiento en régimen de extrapolación fuera de distribución (OOD) valida empíricamente el principio de parsimonia en el modelado físico. Se demuestra que un aumento no justificado en la complejidad topológica de las expresiones conduce sistemáticamente al sobreajuste, manifestado en una divergencia exponencial del error cuadrático medio (MSE) al evaluar dinámicas fuera del dominio de entrenamiento.

Los resultados establecen una clara jerarquía. Los modelos MLP y de regresión polinómica fracasan en la generalización OOD, demostrando su limitación estructural para deducir relaciones algebraicas globales más allá de aproximaciones locales de los datos. En contraste, los regresores simbólicos exhiben capacidad para identificar la expresión analítica exacta. PySR demuestra la mayor robustez topológica y paramétrica, seguido de GPLearn con una tasa de recuperación moderada. QLattice, si bien logra converger hacia la forma funcional correcta en la mayoría de los casos, tiende a generar un colector de soluciones de dimensionalidad excesiva.

En conclusión, la regresión simbólica representa un cambio de paradigma frente a la opacidad del aprendizaje profundo tradicional. Aunque su escalabilidad requiere de un desarrollo continuo de las heurísticas de búsqueda, métodos como PySR demuestran una capacidad excepcional para automatizar la extracción de leyes físicas invariantes puramente a partir de datos empíricos, independizándose de sesgos teóricos a priori.

Referencias

[1] J. Acebes, A. López, and L. Ji. symbolic-physics-discovery. https://github.com/JorgeAcebes/symbolic-physics-discovery, 2026. GitHub repository.

[2] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631. Association for Computing Machinery, July 2019. doi: 10.1145/3292500.3330701. URL http://dx.doi.org/10.1145/3292500.3330701.

[3] K. R. Broløs, M. V. Machado, C. Cave, J. Kasak, V. Stentoft-Hansen, V. G. Batanero, T. Jelen, and C. Wilstrup. An approach to symbolic regression using feyn, 2021. URL https://arxiv.org/abs/2104.05417.

[4] M. Cranmer. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl, May 2023. URL http://arxiv.org/abs/2305.01582. arXiv:2305.01582 [astro-ph, physics:physics].

[5] A. Kaptanoglu, B. de Silva, U. Fasel, K. Kaheman, A. Goldschmidt, J. Callaham, C. Delahunt, Z. Nicolaou, K. Champion, J.-C. Loiseau, J. Kutz, and S. Brunton. Pysindy: A comprehensive python package for robust sparse system identification. *Journal of Open Source Software*, 7(69):3994, Jan. 2022. ISSN 2475-9066. doi: 10.21105/joss.03994. URL http://dx.doi.org/10.21105/joss.03994.

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[7] T. Stephens. *gplearn: Genetic Programming in Python*, 2026. URL https://gplearn.readthedocs.io. Versión 0.4.3. Consultado el 26 de abril de 2026.